

Informe JavaScript - Grupo 3

1. Trabajo realizado por cada participante.....	2
2. Comunicación cliente - servidor.....	3
3. Navegación entre páginas.....	4
4. Gestión de elementos comunes.....	4
5. Consideraciones.....	4

1. Trabajo realizado por cada participante.

Las implementaciones a cargo de Alejandro Cerezo Contreras, alineadas con los acuerdos arquitectónicos del resto del equipo, abarcan:

- Configuración y orquestación del entorno de ejecución mediante *Docker Compose* y *Nodemon* para estandarizar los despliegues formales del grupo. Cabe destacar que todos los compañeros han ido añadiendo endpoints y distintas funcionalidades extra al servidor, puesto que así lo requerían sus implementaciones.
- Implementación integral del *HTML*, *CSS* y *JavaScript* para la visualización y edición dinámica de *Zonas*.
- Implementación integral de la funcionalidad de exportar a *CSV* (Excel).
- Asistencia técnica a Daniil en la lógica y depuración del *Login*.
- Desarrollo de los estilos correspondientes a las vistas de *Zonas* y *Campañas* (parcialmente).
- Construcción de una rama experimental paralela que estructura y unifica el código, y sobre todo *CSS* general de todos los participantes. El *CSS* resultante de esta rama se ha aplicado en *Campañas* y *Zonas*. Pero se podría emplear para el resto de páginas. En esta rama sí se ha tomado especial precaución de seguir mejores prácticas, simplificación y un diseño responsive adaptado a cualquier dispositivo.

Las implementaciones a cargo de Daniil Nemchenko, alineadas con los acuerdos arquitectónicos del resto del equipo, abarcan:

- Implementación integral del *HTML*, *CSS* y *JavaScript* para la visualización y edición dinámica de *Campañas* (parcialmente), adaptación de los estilos a *card-display* (estilo estandarizado universal). Lógica de los filtros de *Campañas* optimizando para que operen directamente sobre la memoria del navegador.
- Implementación integral del *HTML*, *CSS* y *JavaScript* para la visualización del *Login*. (Que incluye componentes comunes adaptados con *Web Component*)
- Implementación de la lógica del *Login* con las comprobaciones y validaciones correspondientes en el lado de servidor (usando cifrado *bcrypt*), creación y manejo del token creado junto con la información adicional necesaria para la identificación del rol en lado de cliente.

Las implementaciones a cargo de Germán Torres Tendero, alineadas con los acuerdos arquitectónicos del resto del equipo, abarcan:

- Página de visualización de todos los colaboradores
- En la página de visualización, dependiendo del rol:
 - Si eres administrador, puedes ver todas los colaboradores de todas las campañas.
- Si eres coordinador, puedes ver los colaboradores de cierta campaña (dada por url) indicada como `idCampania=id de la campaña`.
- Página de edición de un colaborador, solamente accesible si eres manager (coordinador/admin). El id del nuevo colaborador se autogenera en función del último id registrado en la base de datos. También se da la opción de eliminar. Esta función comprueba anteriormente si hay relaciones entre la entidad a borrar y algún voluntario. Si existe, no deja eliminar la entidad, cumpliendo con la integridad de datos.

- Si no eres manager, solamente se muestran los datos sin posibilidad de editar ningún campo.

Las implementaciones a cargo de Candela Dávila Moreno, alineadas con los acuerdos arquitectónicos del resto del equipo, abarcan:

- Implementación integral del *HTML*, *CSS* y *JavaScript* para la pestaña tienda_turnos, incluyendo la visualización por días, el popup de información de voluntarios, y los controles de acceso y visibilidad según rol (ADMIN/COORDINADOR/RESPONSABLE/RESPONSABLE-ENTIDAD).
- Implementación integral del *HTML*, *CSS* y *JavaScript* para la pestaña turno_editar, incluyendo la carga de voluntarios de la entidad responsable, el filtrado, la selección y el guardado de asociaciones turno-voluntario, con validaciones de permisos.
- Implementación integral del *HTML*, *CSS* y *JavaScript* para las pestañas turno_observaciones y turno_observaciones_editar, incluyendo la lectura, edición y guardado de observaciones, además del control de permisos por rol.
- Implementación integral del *HTML*, *CSS* y *JavaScript* para la pestaña turnos_modificar, incluyendo la edición del intervalo de campaña, la creación/eliminación de turnos por día, y la asignación automática de entidad según turnos existentes de la campaña/tienda, con acceso exclusivo para ADMINISTRADOR.

Para probar las funcionalidades descritas se recomienda usar los siguientes usuarios y enlaces:

Usuarios:

- admin@email.com / admin123
- capitan@email.com / capitan123
- responsableasociacionpapilio@email.com / responsableasociacionpapilio123 (responsable de entidad)
- cobos@email.com / cobos123 (responsable de tienda / coordinador)
- abcycolegioelpinar@email.com / abcycolegioelpinar123
- cristobal@email.com / cristobal123

URLs disponibles para probar las funcionalidades:

- Tienda (responsable entidad = responsableasociacionpapilio@email.com, capitan = capitan@email.com, coordinador / responsable de tienda = cobos@email.com) :
http://localhost:3000/html/tienda_turnos.html?idTienda=132&idCampania=1
- Tienda (responsable tienda / coordinador = cristobal@email.com, responsable entidad = abcycolegioelpinar@email.com)

Las implementaciones a cargo de José Carlos Durán Nuño, alineadas con los acuerdos arquitectónicos del resto del equipo, abarcan:

- Implementación integral del HTML, CSS y JavaScript para la vista principal de tiendas (tiendas.js / tiendas.html), incluyendo la generación dinámica de la cuadrícula de tarjetas, los filtros cruzados (por zona, campaña y participación), y la adaptación condicional de la interfaz y botones de acción según el rol del usuario logueado.
- Implementación integral del HTML, CSS y JavaScript para la vista de detalle (info_tienda.js / info_tienda.html), encargada de calcular y desplegar la información completa de localización y asignación jerárquica de la tienda específicamente para la campaña activa de cada momento.
- Implementación integral del HTML, CSS y JavaScript para la vista de edición (editar_tienda.js / editar_tienda.html), desarrollando la lógica de selectores dinámicos dependientes. Esto incluye el filtrado automático de Capitanes y Responsables condicionados a la zona geográfica de la tienda seleccionada, garantizando la integridad de las asignaciones, con acceso restringido de forma estricta al rol ADMINISTRADOR.
- Refactorización de todo el módulo de tiendas para utilizar exclusivamente la API nativa del DOM (creación de nodos programática sin uso de innerHTML), blindando la aplicación contra inyecciones XSS y estandarizando el código a las buenas prácticas exigidas.

2. Comunicación cliente - servidor.

De acuerdo con la estructura API REST y helpers consolidados por todo el equipo para el proyecto:

- En la sección de Zonas, el cliente carga información asíncrona apuntando a funciones genéricas en *api.js* (ej. *getZones()*). Dicha estructura maneja inherentemente el **fetch** hacia el servidor Node, autoinyectando el JWT de sesión exigido, facilitando así la comunicación segura e ininterrumpida con los joins anidados de Supabase.
- En el módulo Login el cliente envía las credenciales mediante una petición asíncrona **post** (a través de *api.js*), el servidor Node los valida y envía JWT en el caso de autenticación exitosa. Este token se guarda en *sessionStorage* junto con la información adicional para comprobaciones internas del rol en el lado de cliente.
- En la sección de *Campañas* se recuperan los datos de las campañas con el **fetch** (usando *getCampañas()* de *api.js*) y el filtrado se realiza en el navegador sin necesidad de peticiones adicionales.
- La comunicación cliente-servidor en la gestión de turnos se ha resuelto mediante llamadas **fetch** desde el frontend (JS) a la API del backend, enviando y recibiendo JSON con cabeceras de autorización, de forma que las vistas obtienen los datos de turnos, campañas, voluntarios y observaciones, y ejecutan acciones de creación/edición/eliminación a través de endpoints específicos (/api/tienda_turnos, /turnos, /api/turno_borrar_dia, /api/turno_guardar_voluntarios, etc.), manteniendo la lógica de permisos y roles en el cliente y validaciones clave en el servidor.
- La comunicación cliente-servidor en el módulo de tiendas y su asignación de campañas se ha abordado mediante peticiones asíncronas **fetch** hacia los endpoints REST de la API (/api/tiendas, /api/cps, /api/cadenas, etc.), enviando el token JWT de sesión en las cabeceras

de autorización. Para optimizar los tiempos de carga en las vistas de detalle y edición, se ha implementado la resolución concurrente de múltiples promesas (Promise.all), recuperando los catálogos de datos simultáneamente. Las operaciones de actualización utilizan el método PUT enviando el payload en formato JSON. Como medida de seguridad y rendimiento, el filtrado base de las tiendas según la demarcación del usuario (ej. COORDINADOR viendo solo su zona) se ejecuta directamente en la consulta a la base de datos desde el backend (Supabase), delegando al cliente únicamente el renderizado final del DOM.

3. Navegación entre páginas.

La navegación entre las diferentes vistas del proyecto se orquesta bajo un enfoque MPA (Multi-Page Application), gestionando los redireccionamientos mediante enlaces en el HTML y saltos programáticos en JavaScript.

Para estructurar la interfaz de usuario de forma eficiente y evitar redundancias, el equipo ha adoptado el uso de un patrón modular para la cabecera y el pie de página. Concretamente, se ha integrado en las distintas páginas el **Web Component** base `<include-html>` desarrollado por Candela. Su implementación en el frontend permite inyectar dinámicamente estos bloques de navegación y, a su vez, resaltar en el menú en qué vista se encuentra el usuario de forma sencilla parametrizando un atributo.

4. Gestión de elementos comunes.

Como mencionamos con anterioridad, la navegación y lógica de vistas del proyecto se orquesta bajo un enfoque *MPA (Multi-Page Application)*. Dado este enfoque, era crítico diseñar una solución para evitar duplicar el código del menú de navegación en los múltiples archivos `.html`.

Para estructurar la interfaz de usuario de forma eficiente y compartir la cabecera y el pie de página, el equipo ha implementado un sistema de inyección dinámica mediante el Web Component `<include-html>` desarrollado por Candela. Se trata de un Custom Element de JavaScript que permite modularizar la aplicación de forma nativa. Al utilizar `<include-html src="componentes/header.html">` en cualquier página de la aplicación, este se encarga internamente de solicitar mediante *fetch* el documento común y renderizarlo en el DOM actual. La implementación del frontend no solo carga el *HTML* en bruto, sino que permite recibir parámetros interactivos. Por ejemplo, al definir un parámetro `active="zonas"`, el componente identifica automáticamente qué elemento del menú de navegación base debe adquirir la clase CSS de resaltado, marcando al usuario visualmente la página activa sin necesidad de alterar el archivo de origen.

5. Consideraciones.

Instrucciones para poder ver la aplicación:

1. Configurar variables de entorno

- Crea un archivo .env en la raíz del proyecto:
- Edita .env con estas credenciales de Supabase y la URL de la API:

SUPABASE_URL=https://ybkoqvyakvzpvsgoaacx.supabase.co

SUPABASE_KEY=sb_publishable_GWP4aBzP2Jt4DItKSNqulQ_FIcCLQHE

SUPABASE_SERVICE_KEY=sb_secret_wSJ0PpoPUOkgw-1zVfTBDg_ARMJqI9X

API_URL=<http://localhost:3000>

2. Levantar la aplicación:

- docker-compose up -d

La aplicación estará disponible en:

- Frontend: <http://localhost:3000>
- API: <http://localhost:3000/health>

Para acceder a los endpoints se usará el prefijo /api y para acceder a las páginas html el prefijo /html